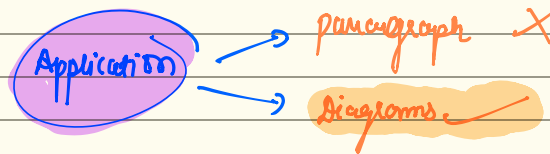


UML Diagrams.

→ UML diagrams:- Unified modelling language diagrams → a standard way to visualize, design and document software systems.

Why UML?

- Understand how a system works
- Communicate ideas with your team
- Plan classes, flows, architecture
- Reduce bugs by thinking before coding



UML diagrams

Structural
(Static)

7

Class Diagram

Behavioural
(Dynamic)

7

Sequence Diagram

Only 2 imp. UML diagrams for entry level SW engineering

1 Structural Diagrams – What the system is

These describe the static structure of your system.

◆ Class Diagram

Shows classes, attributes, methods, and relationships.

➡ Most common UML diagram.

◆ Object Diagram

Shows objects and values at a particular moment (snapshot).

◆ Component Diagram

Shows how components/modules interact.

◆ Deployment Diagram

Shows hardware nodes → servers, devices, etc.

◆ Package Diagram

Shows how classes/packages are grouped.

These describe how things behave, interact, or flow.

◆ Use Case Diagram

Shows actors (user/system) and how they interact with the system.

◆ Sequence Diagram

Shows how objects communicate in order (time-based).

◆ Activity Diagram

Flowchart of actions, decision, loops → like logic flow.

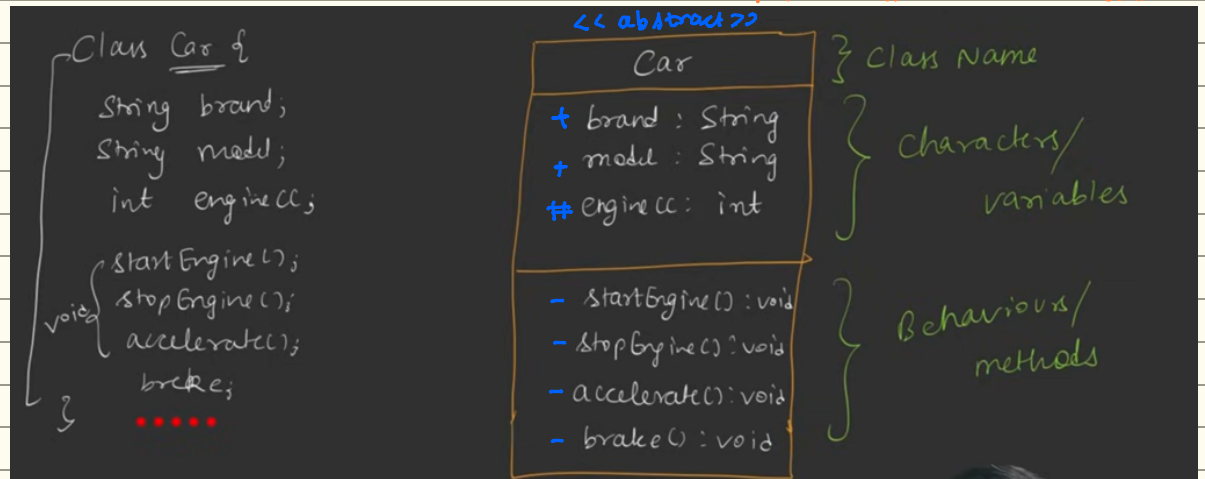
◆ State Machine Diagram

Shows states of an object and transitions.

◆ Communication Diagram

Similar to sequence but focuses on message passing.

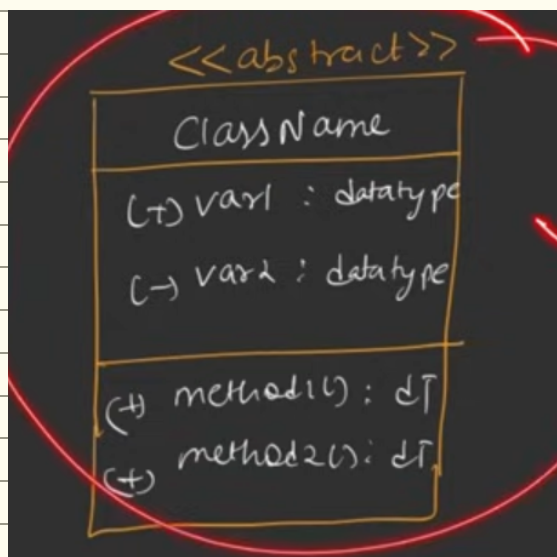
→ **CLASS DIAGRAM:-** Shows classes, attributes, methods and relationships.
 ↳ if this is a abstract class.



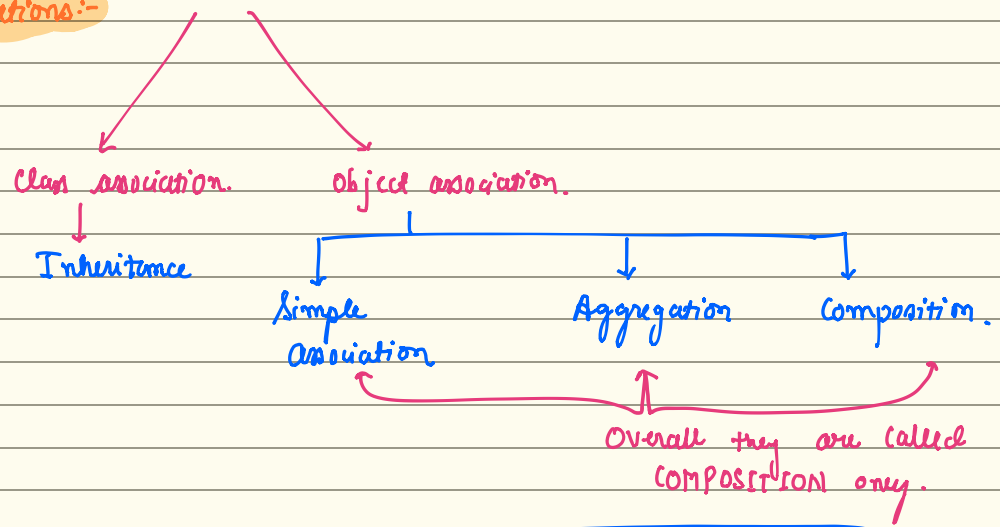
→ **Access modifiers:-**

Representation in UML diagram.

		Within class	from child class	outside the class
(+)	Public	(✓)	(✓)	(✓)
(#)	Protected	(✓)	(✓)	(x)
(-)	Private.	(✓)	(x)	(x)



→ Associations:-



→ Inheritance:- Inheritance is a (is-a) relationship.

```
Class A {
```

```
    method 1();
```

```
}
```

```
Class B extends A {
```

```
    method 2();
```

```
}
```

```
main() {
```

```
    B* b = new B();
```

```
    b → method 1();
```

```
    b → method 2();
```

```
}
```



Cow is-a Animal

∴ " → "

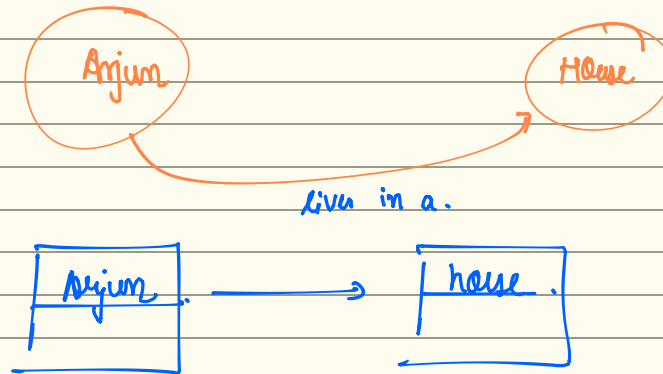
is used to represent inheritance in UML diagram.

→ Composition :- (has a) relationship.

→ Simple Association :- A Simple Association in UML is the most basic type of relationship between two classes. It simply means:

👉 One class knows about or interacts with another class.

There is no ownership, no lifecycle dependency, and no special constraints—just a plain connection.



→ Aggregation Aggregation in UML is a "has-a" relationship where one class contains or owns other objects — but the lifecycles are independent.

It is a weak form of ownership.

✅ Simple Definition

Aggregation means:

- 👉 A whole has parts
- 👉 But the parts can exist independently of the whole
- 👉 If the whole is destroyed, the parts still exist

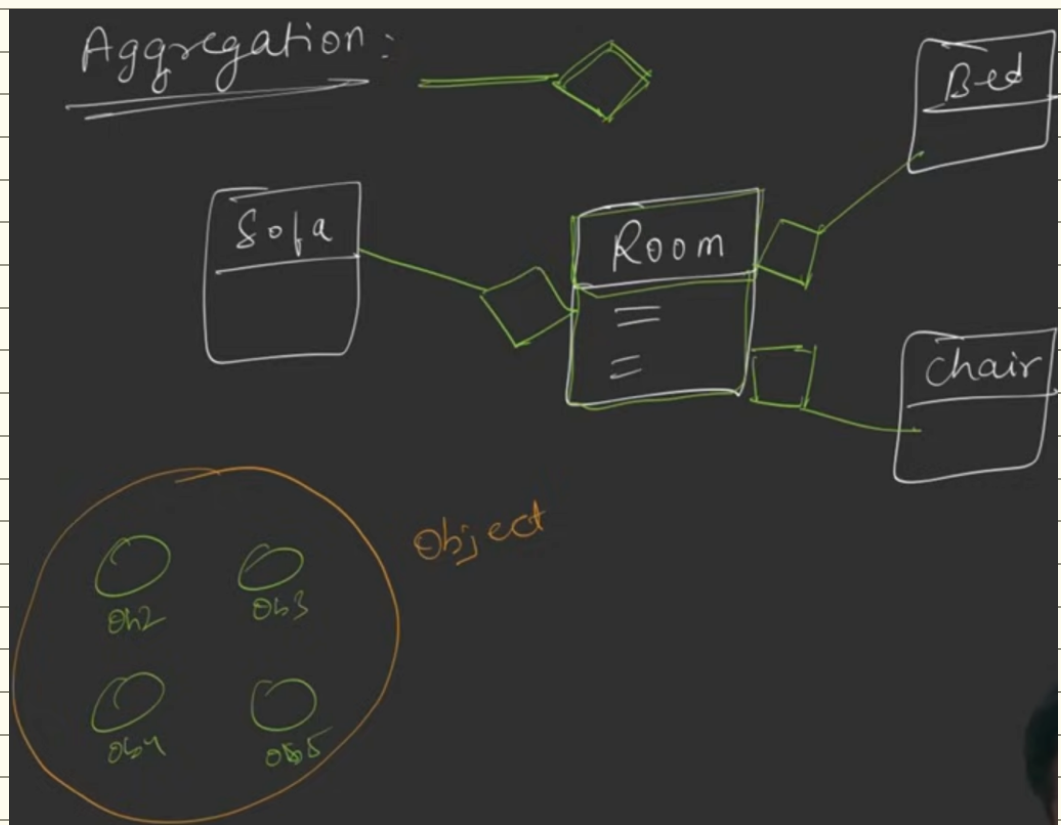
◆ UML Symbol

Aggregation is shown using a **hollow diamond** (◊) near the "whole".

lua

📄 Copy code

Department ◊----- Professor



→ Composition:-

Composition in UML is a strong "has-a" relationship where one class owns another class completely. It is the strongest form of association.

✓ Simple Definition

Composition means:

- ✎ The whole owns the part
- ✎ The part cannot exist without the whole
- ✎ If the whole is destroyed, the part also gets destroyed
- ✎ Strong lifecycle dependency

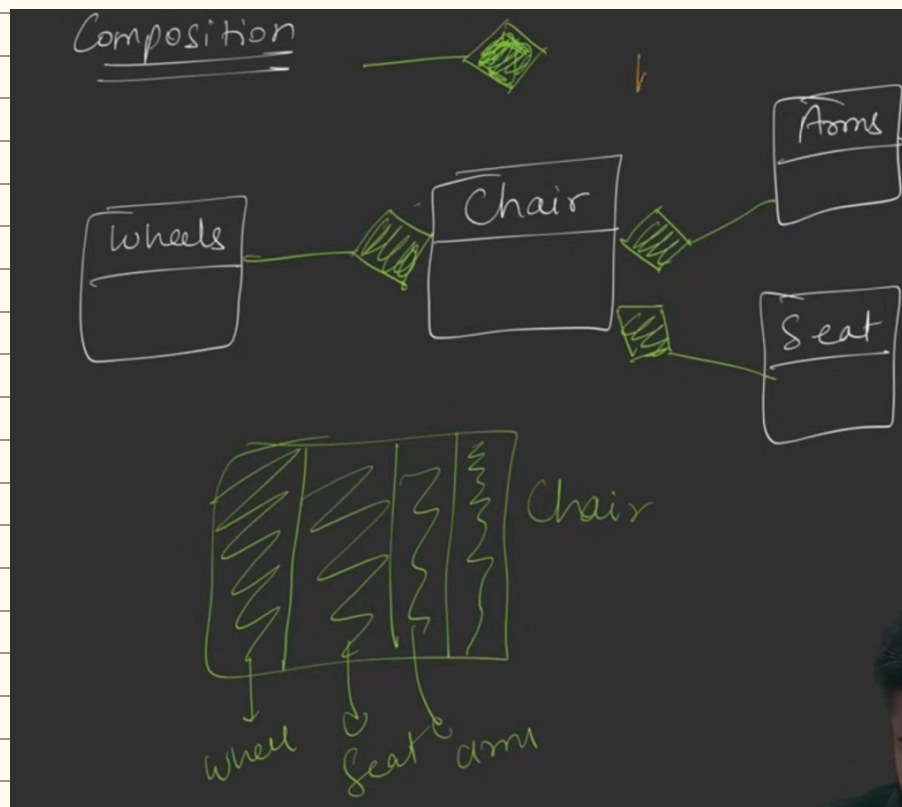
UML Symbol

Composition is shown with a **filled (solid) diamond** next to the "whole".

nginx

Copy code

House ◆ Room



✓ Java Code Example (Composition)

A `Car` owns an `Engine` — the `Engine` cannot exist without the `Car`.

java

[Copy code](#)

```
class Engine {
    void start() {
        System.out.println("Engine started");
    }
}

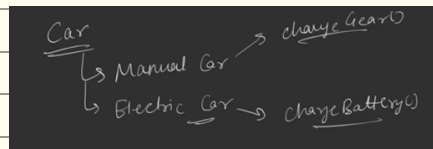
class Car {
    private Engine engine; // Composition: Car owns Engine

    public Car() {
        this.engine = new Engine(); // Engine is created inside Car
    }

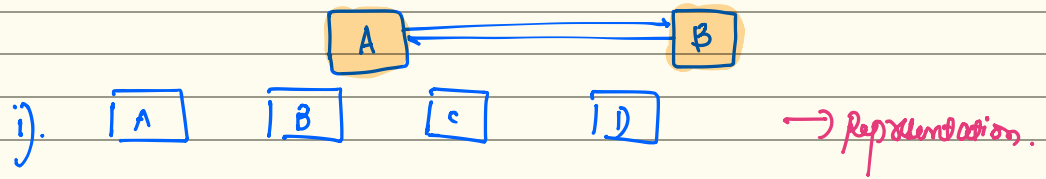
    public void startCar() {
        engine.start();
        System.out.println("Car is running");
    }
}

public class Main {
    public static void main(String[] args) {
        Car car = new Car();
        car.startCar();
    }
}
```

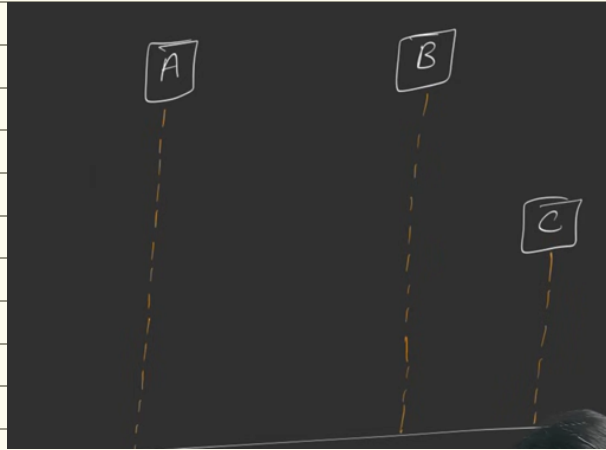
Ex:- UML diagram for:-



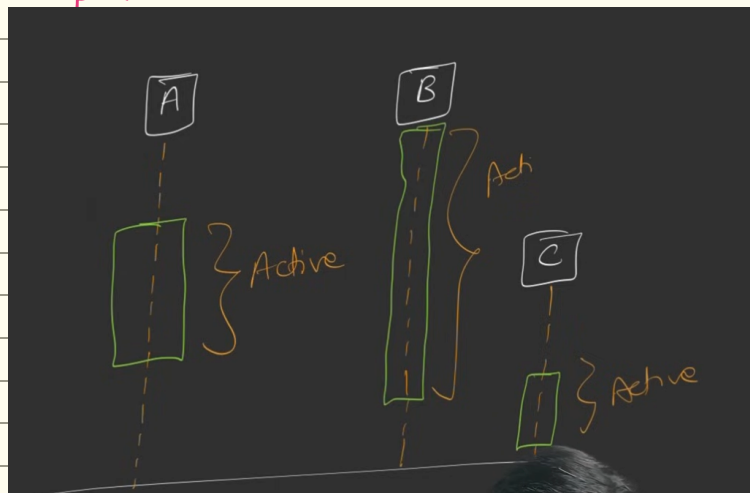
Sequence Diagram :- Communication / instruction.



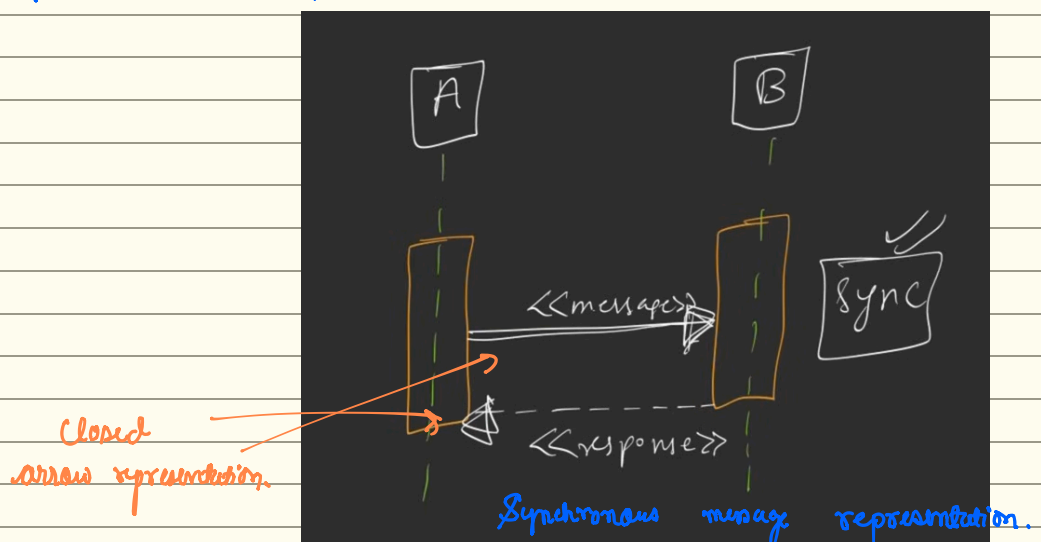
ii). Lifetime:-

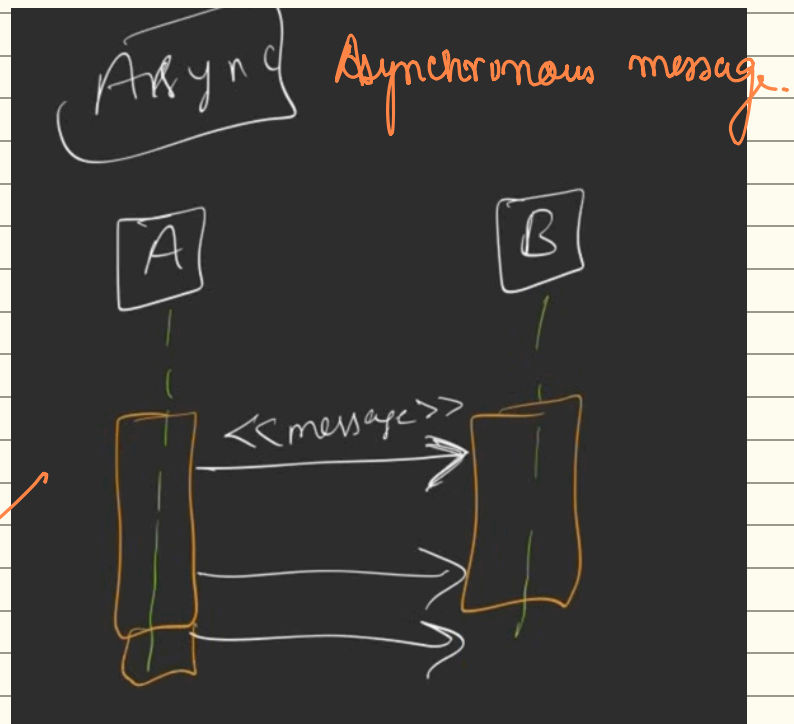


(iii). Activation Bar:-

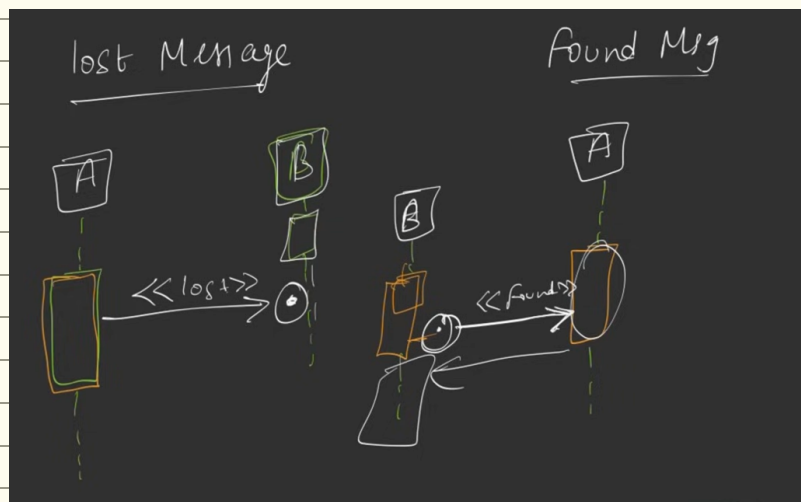
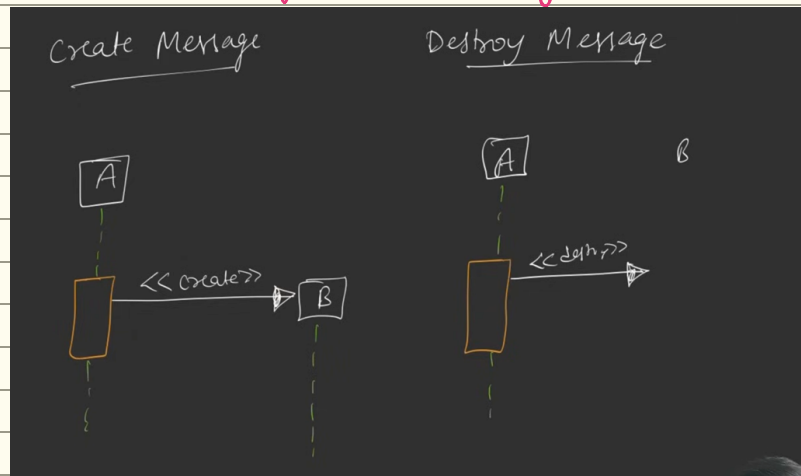
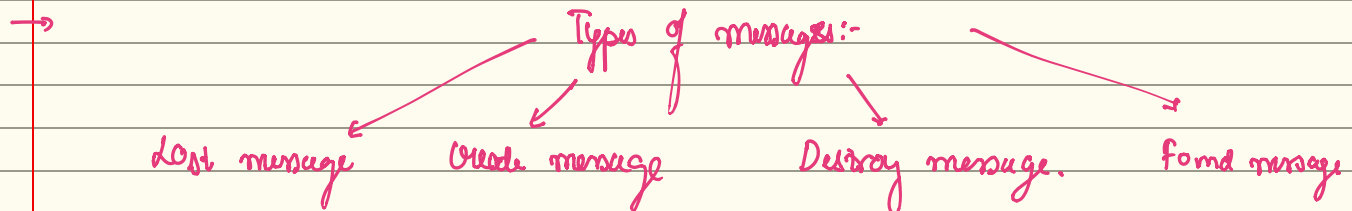


(iv). Message
↓
Asynchronous Synchronous.

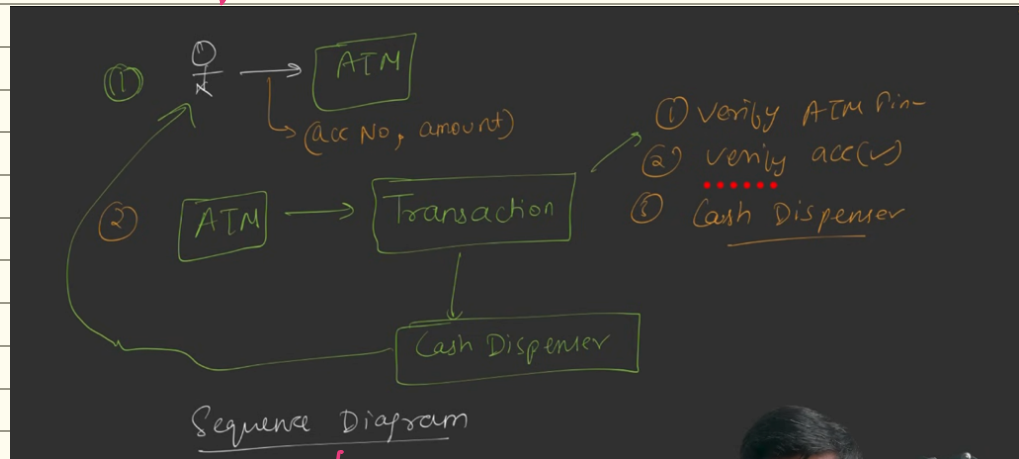




Open
lives.



→ How to draw sequence diagram:-



Steps:-

(i) Use Case

(ii) Objects.

→ ATM

→ User

→ Transaction.

→ Account

→ Cash Dispenser.

(iii). Draw Sequence Diagram.

